

 No description has been provided for this image

Table of Contents & Jumplinks

Coding Package Introductions

- [Matplotlib background](#)
- [package imports](#)
- [create fake datasets](#)
- [matplotlib coding basics](#)

Basic Data Visualizations

- [Bar plots](#)
- [Box plots](#)
- [Scatter plots](#)
- [Line/timeseries plots](#)
- [Histograms](#)
- [image data](#)
- [multiple plots per figure](#)

Introduction to Matplotlib

 No description has been provided for this image

matplotlib is a plotting library for Python.

Pros:

- Huge amount of functionality/options.
- Works with numpy arrays and python lists.
- Comes with many prepackaged Python distros (anaconda, WinPython, etc.).
- Easily saves plots to image (.png, .bmp, etc.) and vector (.svg, .pdf, etc.) formats.
- Has an excellent set of examples (with code) at the [matplotlib gallery](#)
- Shares many syntactic conventions with Matlab.

Cons:

- Slow for rapidly updating plots.
- 3D plotting support is not great.
- Documentation is not always useful.

- Essentially has two interfaces. One is intended to be close to Matlab, the other is object oriented. You will find examples that assume one or the other, but rarely the one you are after.
- Shares many syntactic conventions with Matlab.

Resources:

- [cheat sheets](#)
- [youtube tutorials](#)

```
In [1]: # Import numpy and pyplot
import matplotlib.pyplot as plt
import numpy as np
```

Creating Fake Data

The first thing we need to create some fake data so we have something to plot! Lets create fake:

- timeseries data (times and values)
- Categorical and Numeric data (categories and values)

```
In [2]: # timeseries
times = np.arange(0, 10, 0.01) # make evenly spaced points between 0 and 10

# lets give ourselves a few different 'signals' to plot
sin_values = np.sin(times)
cos_values = np.cos(times)
osc_values = np.sin(2 * np.pi * times) * np.exp(-0.5 * times) # some function
```

```
In [3]: # categorical data

# Adoptable cats!
# categories
cat_ids = np.array([0, 1, 2, 3, 4, 5, 6])
cat_names = ['Ace', 'Bonsi', 'Comet', 'Dazzler', 'Espresso', 'Frank', 'Gertrude']

# some values per cat
adopt_apps = np.array([3, 2, 7, 13, 5, 4, 10])
age_years = np.array([3, 5, 1, 9, 5, 2, 8])
```

Matplotlib Basics

We will be using `pyplot.subplots ( plt.subplots() )` to create our beautiful data visualizations. Calling `plt.subplots()` gives back two objects that are

conventionally called a "figure" and an "axes". To represent these we will create two variables, **fig** and **ax**.

fig: You can think of the figure as the frame of your plot.

- You can resize, reshape the frame but you cannot draw on it.
- On a single notebook or a script, you can have multiple figures.
- Each figure can have multiple subplots.

ax: an axes is your metaphorical canvas to draw on. Or rephrasing, it is the blank sheet you can plot and hold your data.

- An axes object can only belong to one figure.

No description has been provided for this image

No description has been provided for this image

Matplotlib Code Basics

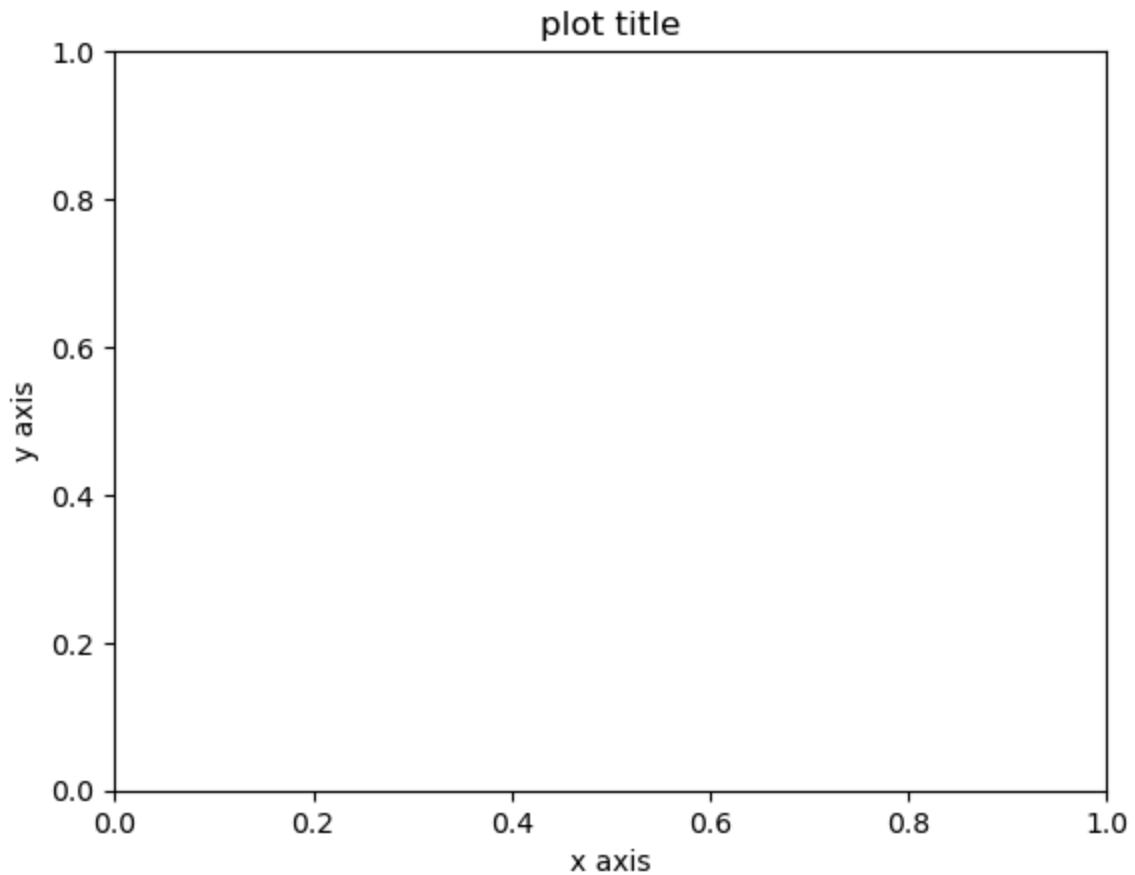
- create a figure: `fig, ax = plt.subplots()`
 - Adjust figure size: `fig, ax = plt.subplots(figsize = (horizontal, vertical))`
 - Multiple plots per figure: `fig, ax = plt.subplots(n_rows, n_columns)`
 - Note!: we will cover this more in-depth later!
- add title: `ax.set_title('plot title')`
- set x label: `ax.set_xlabel('x label')`
- set y label: `ax.set_ylabel('y label')`

NOTE: Later (tomorrow) you will learn Seaborn, which is another plotting package. Seaborn is essentially a wrapper for matplotlib there are many matplotlib conventions and functions that we will use on almost all of our plots regardless of whether they are seaborn or matplotlib plots

In [4]: *# Lets practice by creating a blank plot!*

```
fig, ax = plt.subplots()
ax.set_title('plot title')
ax.set_xlabel('x axis')
ax.set_ylabel('y axis')
```

Out[4]: Text(0, 0.5, 'y axis')



Bells and Whistles

The Axes object (`ax`) that we created above provides many methods to help us annotate our plots:

Tick Marks:

- `ax.set_xticks`
- `ax.set_xticklabels`

Error Bars and Confidence Intervals

- `ax.errorbar()`

Many more options are available here -- you can customize the border, axis ticks, tick labels, line styles, fonts, spacing, etc. The [documentation](#) and examples available online are good, so usually a google search will get you answers quickly.

Bar Plots

`.bar` will create a basic bar plot that you will have to add titles and axes labels to.

Basic code:

```
fig, ax = plt.subplots()

ax.bar(x values, y values)

# for bar charts, we often want to customize the x axis ticks and
labels
ax.set_xticks(x values)
ax.set_xticklabels( x labels);

# set axis labels
ax.set_xlabel('x axis')
ax.set_ylabel('y axis')
```

```
In [5]: #Lets make a bar plot!

# we will use our fake categorical data for bar plots

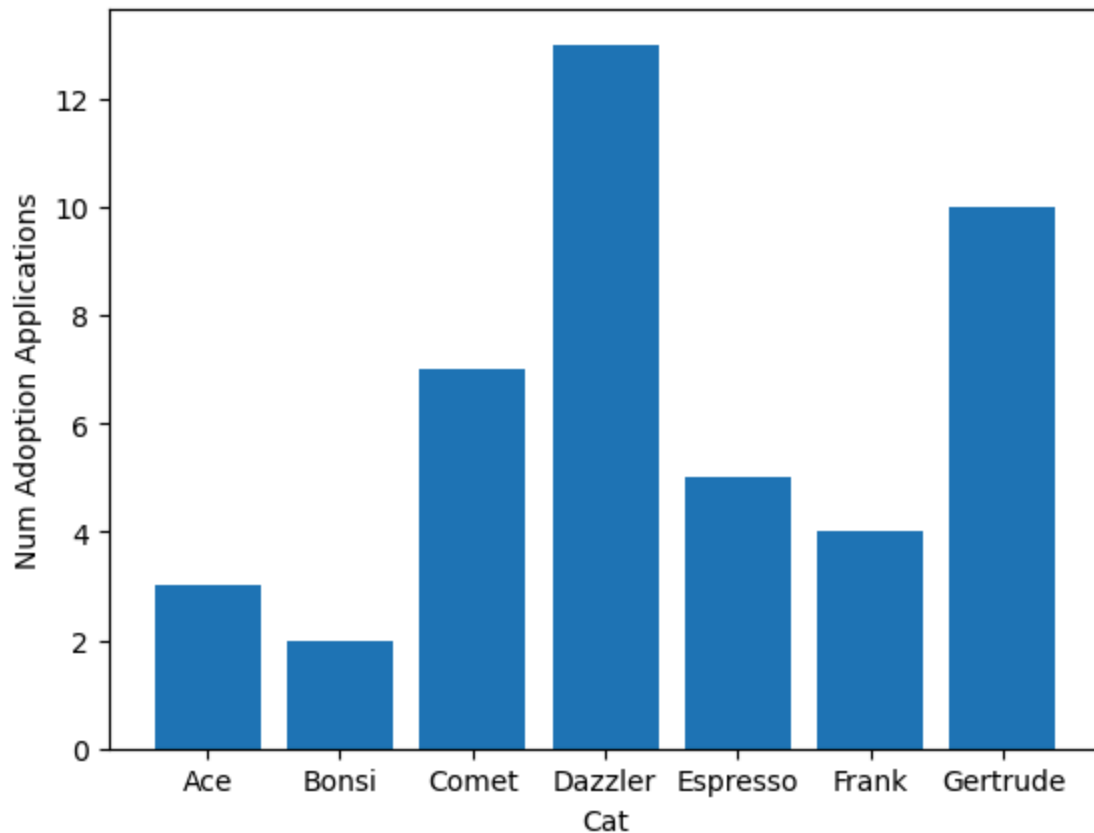
fig, ax = plt.subplots()

ax.bar(cat_ids, adopt_apps)

# for bar charts, we often want to customize the x axis ticks and labels
ax.set_xticks(cat_ids)
ax.set_xticklabels(cat_names);

# set axis labels
ax.set_xlabel('Cat')
ax.set_ylabel('Num Adoption Applications')
```

```
Out[5]: Text(0, 0.5, 'Num Adoption Applications')
```



Box Plots

`.boxplot` will create a basic boxplot that you will have to add titles and axes labels to.

Basic code:

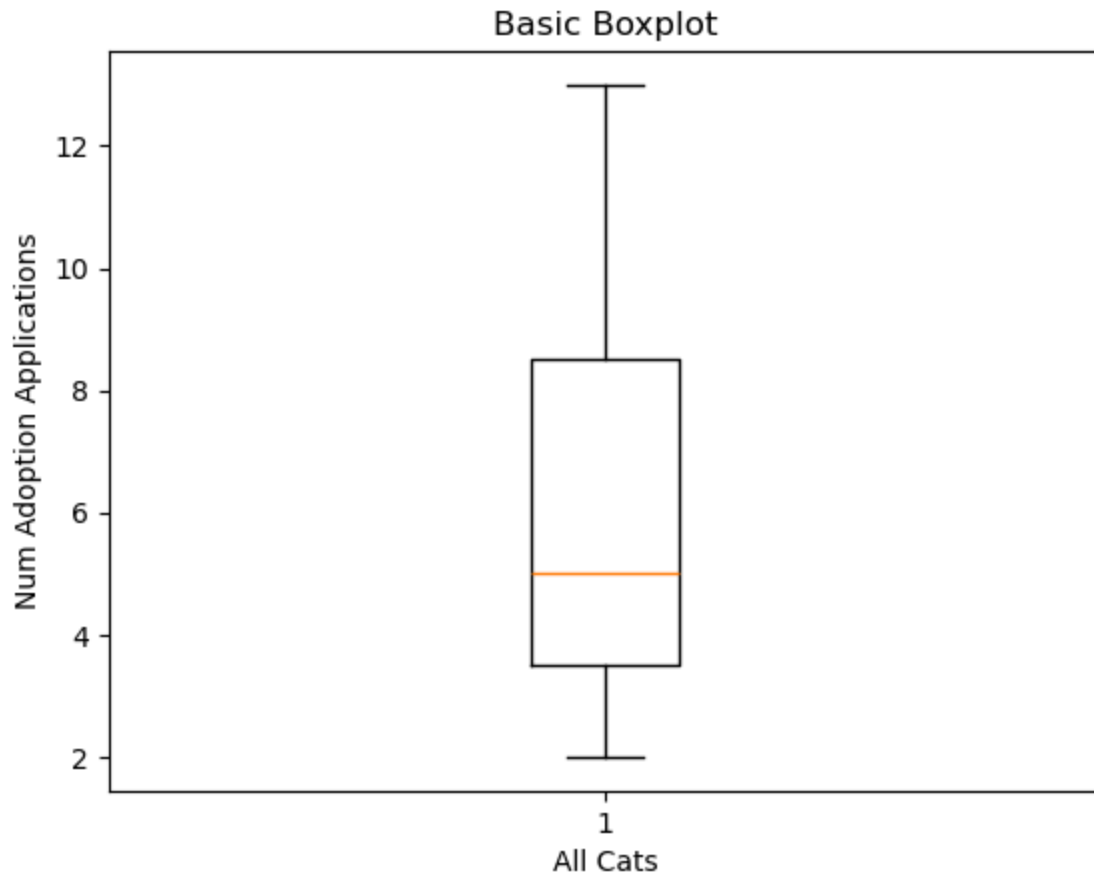
- vertical: `plt.boxplot(values of interest)`
- horizontal: `plt.boxplot(values of interest, vert=False)`

```
In [6]: # Lets plot the distribution adoption applications for our sweet little cats
        # using a boxplot!
```

```
fig, ax = plt.subplots()
ax.boxplot(adopt_apps)
ax.set_title('Basic Boxplot')

# axis labels
ax.set_ylabel('Num Adoption Applications')
ax.set_xlabel('All Cats')
```

```
Out[6]: Text(0.5, 0, 'All Cats')
```



Scatter Plots

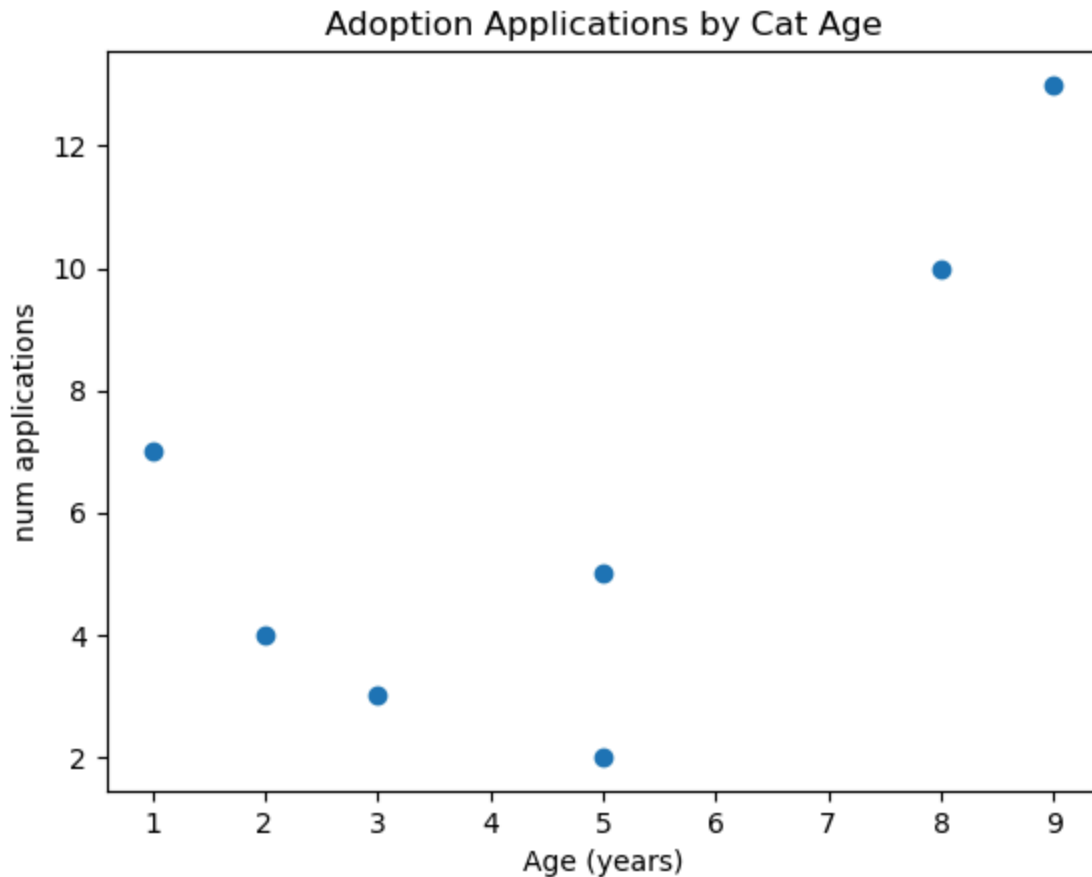
Matplotlib has a built in scatter plot function: `.scatter()`

Basic code

```
ax.scatter(x = x values,  
           y = y values)
```

```
In [7]: # Now lets plot age vs number of applications  
fig, ax = plt.subplots()  
  
ax.scatter(x = age_years,  
           y = adopt_apps)  
  
ax.set_title('Adoption Applications by Cat Age')  
ax.set_xlabel('Age (years)')  
ax.set_ylabel('num applications')
```

```
Out[7]: Text(0, 0.5, 'num applications')
```



Lineplots

`.plot()` offers very basic line plotting

Basic code: `plt.plot(x, y)`

Bells and Whistles

```
fig, ax = plt.subplots()
```

```
# plot multiple lines
```

```
# specify line color, line thickness and give a label for each  
line (will be displayed in the legend)
```

```
ax.plot(x, y1, color='red', linewidth=.5, label='label text') #  
line 1
```

```
ax.plot(x, y2, color='blue', linewidth=2, label='label text') #  
line 2
```

```
# Add a title at the top of the plot  
ax.set_title('title here')
```

```
# Add x/y axis labels  
ax.set_xlabel('x label')  
ax.set_ylabel('y label')
```



```
# set the viewable window of the x or y axis
# ax.set_xlim(lower limit, upper limit)
# ax.set_ylim(lower limit, upper limit)

# Add a legend to plot the line labels assigned above!
ax.legend();
```

Exercise 1.1

Make a simple lineplot of the of `osc_values` by `times`

- set the title to "A Damped Oscillator"
- set the x and y axis to 'Time (s)' and 'Amplitude'

```
In [8]: # SOLUTION 1

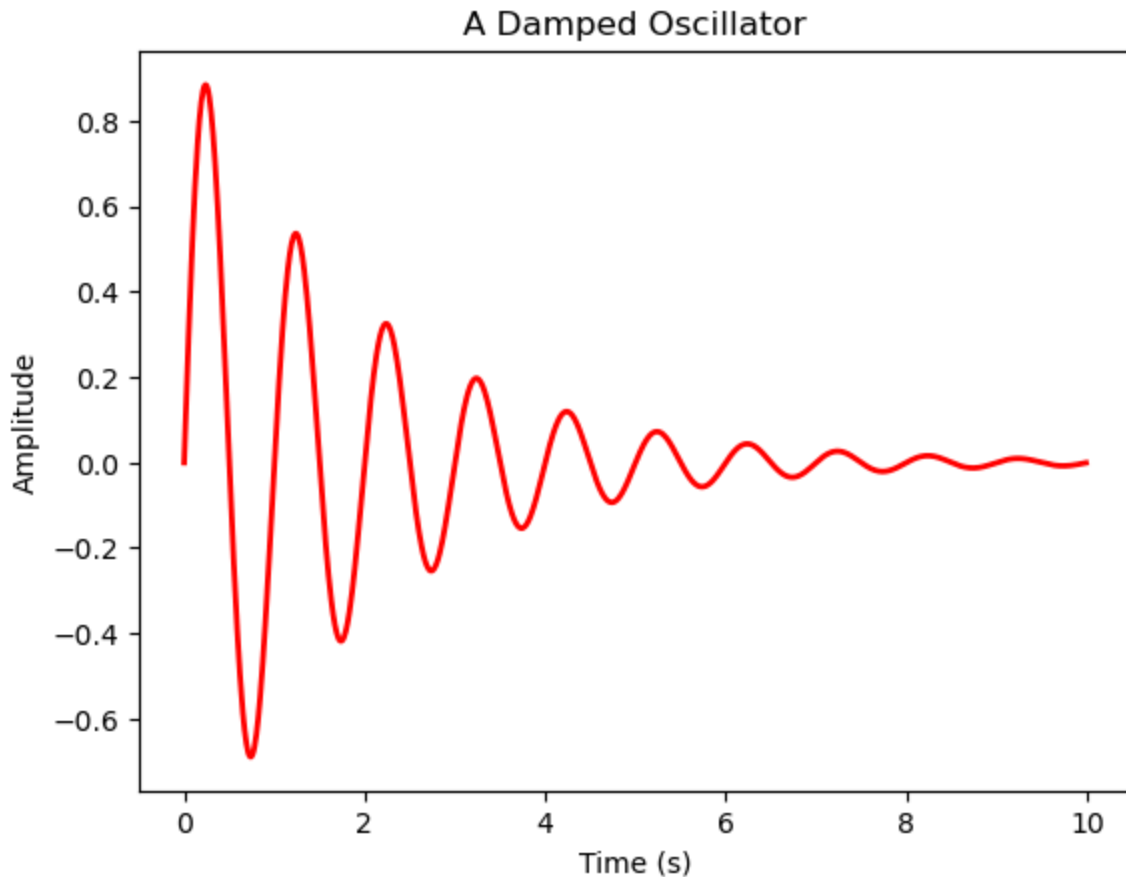
fig, ax = plt.subplots()

# Plot our data!
ax.plot(times, osc_values, color='red', linewidth=2);

# Add a title at the top of the plot
ax.set_title('A Damped Oscillator')

# Add x/y axis labels
ax.set_xlabel('Time (s)')
ax.set_ylabel('Amplitude')
```

```
Out[8]: Text(0, 0.5, 'Amplitude')
```



Exercise 1.2

Make a lineplot that includes the `sin_values` and `cos_values` by `times` .

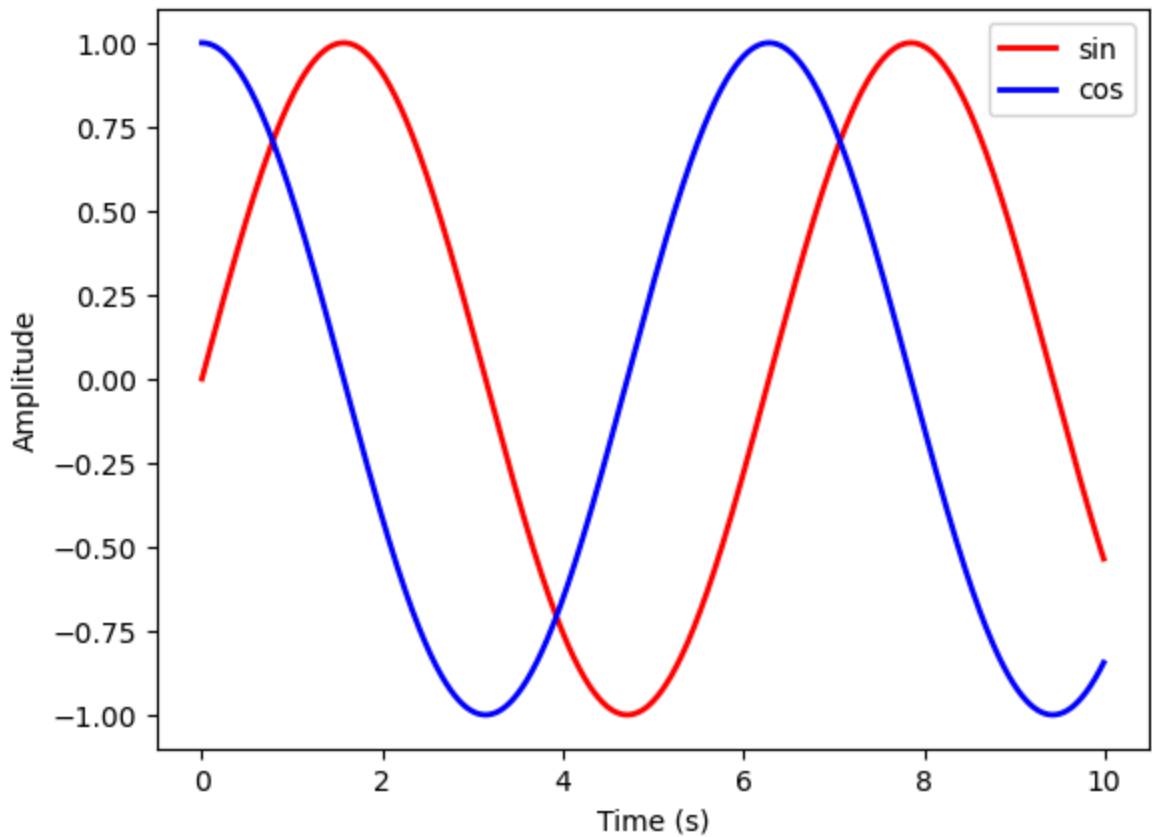
- make sin to red and give 'sin' as label name
- make cos blue and give 'cos' as label name
- add same x,y axis labels as above

```
In [9]: # SOLUTION 2
fig, ax = plt.subplots()

ax.plot(times, sin_values, color='red', linewidth=2, label='sin')
ax.plot(times, cos_values, color='blue', linewidth=2, label='cos')

# Add x/y axis labels
ax.set_xlabel('Time (s)')
ax.set_ylabel('Amplitude')

# Add a legend. Note the `label` argument to ax.plot!
ax.legend();
```



Histograms

histograms are relatively simple with matplotlib using `.hist`

frequency: `plt.hist(values of interest, bins = int)`

density: `plt.hist(values of interest, bins = int, density = True)`

```
In [10]: # Lets practice making a histogram with the oscillation values
```

```
fig, ax = plt.subplots()
ax.hist(osc_values, bins=30);
```

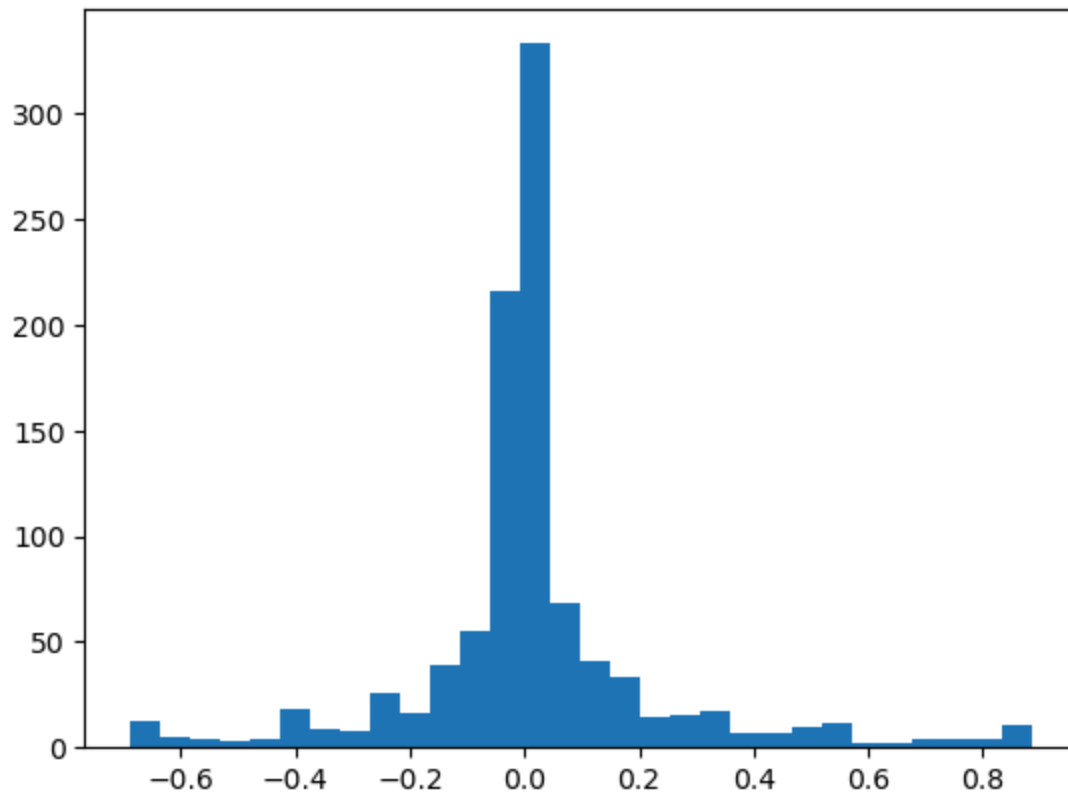


Image Data

Any time we have a 2D array of values, it may be useful to visualize the array as an image. Matplotlib gives us many options to control how data values are converted to image pixel colors. This process is very simple with matplotlib, just use `.imshow()`

 No description has been provided for this image

Basic code: `ax.imshow(2d_image_array_data)`

Helpful Parameters

- `cmap= string` : The Colormap instance or registered colormap name used to map scalar data to colors. This parameter is ignored for RGB(A) data. Default: 'viridis' For more colormaps check out the [colormap documentation](#))
- `clim =(min,max)` : Set the color limits of the current image. If either vmin or vmax is None, the image min/max respectively will be used for color scaling.

In [11]: *# First, make an interesting 2D array of values*

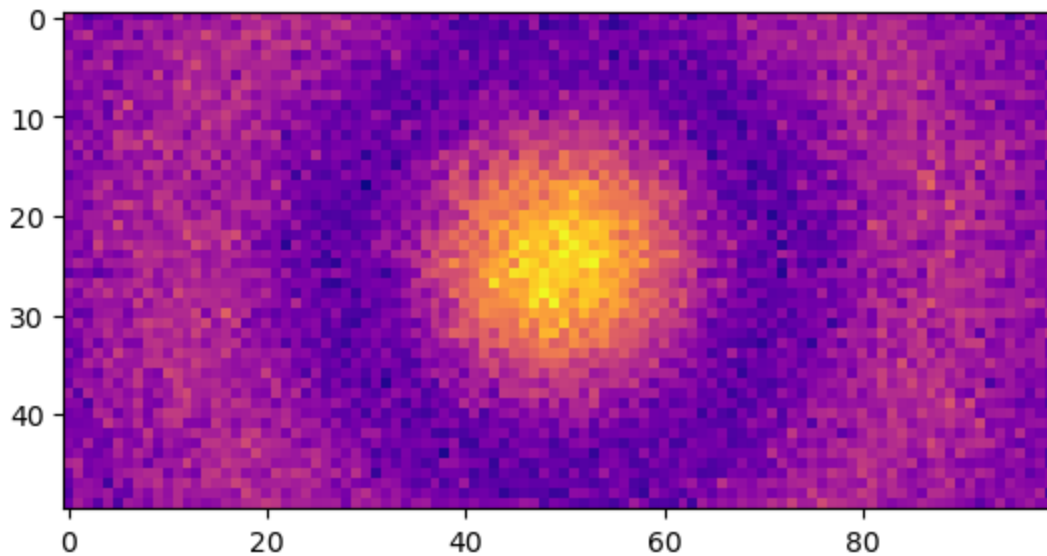
```
x = np.linspace(-10, 10, 100)
y = np.linspace(-5, 5, 50)
r = (y[:, None]**2 + x[None, :]**2)**0.5
data = np.sin(r) / r + np.random.normal(size=(50, 100), scale=0.1)
```

```
print("Array shape:", data.shape)
```

Array shape: (50, 100)

In [12]: *# show as an image*

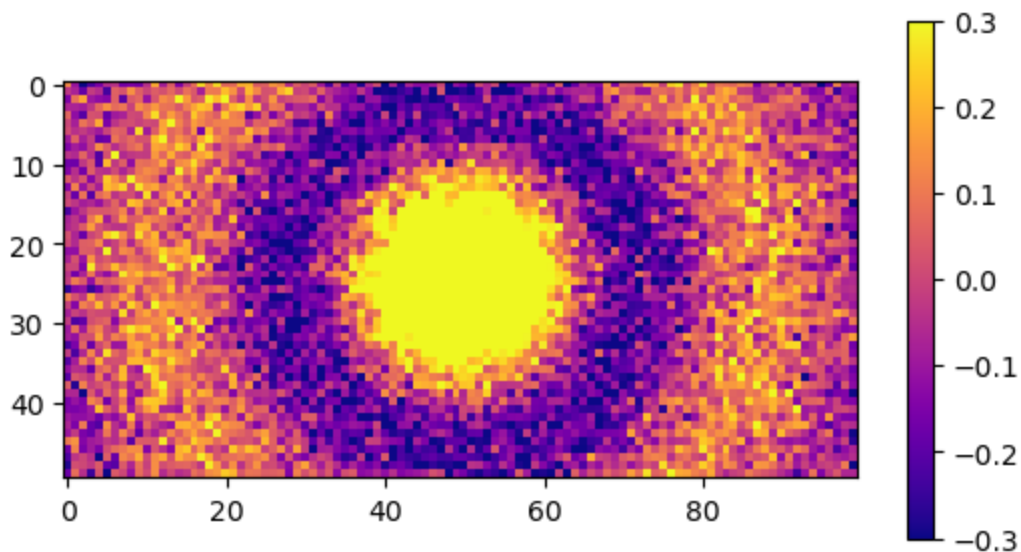
```
fig, ax = plt.subplots()
ax.imshow(data, cmap='plasma');
```



In [13]:

```
fig, ax = plt.subplots()
image = ax.imshow(data, cmap='plasma', clim=(-0.3, 0.3))

# Add a colorbar using the previously-returned image object (which carries i
fig.colorbar(image, ax=ax, shrink=0.7);
```



Multi Plot Figures

Figures with Subplots

A given figure can have more than one axis. The subplot command, which we used above, generates a single axis by default. But we can specify the number of axes that we want..

 No description has been provided for this image

Complex Figures with Gridspec

`Gridspec` is useful when you have uneven subplots. It can get tricky for more complex plots, so first try to use `ax.subplots()` (like in the previous examples) if possible.

 No description has been provided for this image

Subplot

`.subplots`: creates a figure and a set of subplots. This utility wrapper makes it convenient to create common layouts of subplots, including the enclosing figure object, in a single call.

General code example:

```
fig, axes = plt.subplots(nrows=2, ncols=1)
axes[0].plot()
axes[1].plot()
```

Optional Parameters for subplots call:

- `sharex/sharey = bool` forces plots to display the same range along the x-axis or y-axis depending upon which argument you call. Default = False

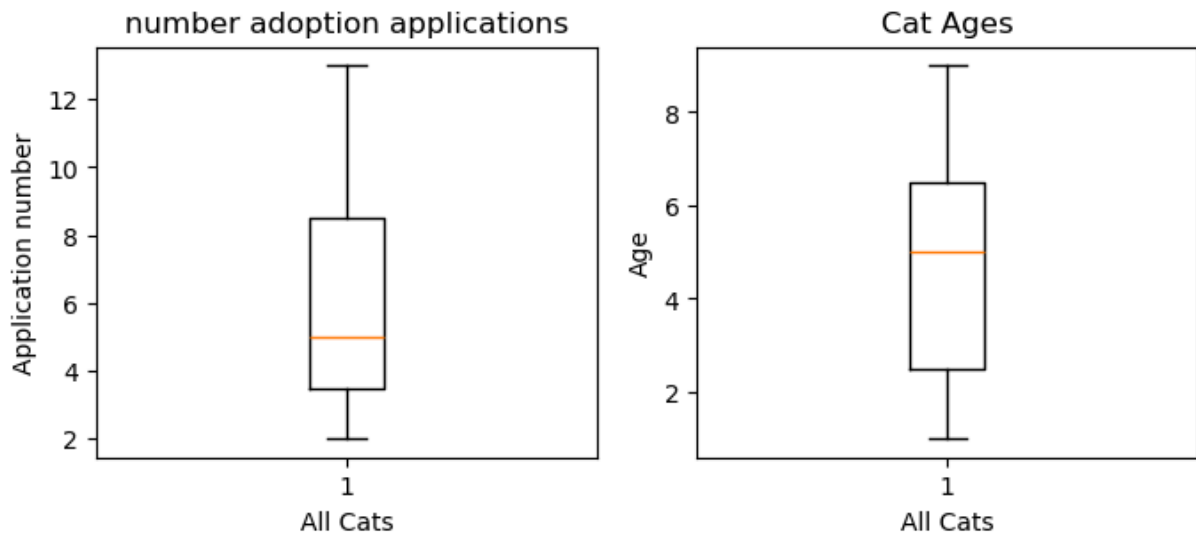
```
In [14]: # Lets create 2 basic boxplots of number of applications and cat age and put
# in the same figure:
```

```
fig, axes = plt.subplots(nrows=1,ncols=2, figsize=(8,3))

axes[0].boxplot(adopt_apps)
axes[0].set_title('number adoption applications')
axes[0].set_ylabel('Application number')
axes[0].set_xlabel('All Cats')

axes[1].boxplot(age_years)
axes[1].set_title('Cat Ages')
axes[1].set_ylabel('Age')
axes[1].set_xlabel('All Cats')
```

```
Out[14]: Text(0.5, 0, 'All Cats')
```



gridspec: matplotlib

.subplots: creates a figure and a set of subplots. This utility wrapper makes it convenient to create common layouts of subplots, including the enclosing figure object, in a single call.

General code example: `

```
fig = plt.figure()
gridspec_fig = fig.add_gridspec(nrows=2, ncols=2)
ax0 = fig.add_subplot(gs[0, 0]) # Top left corner
ax1 = fig.add_subplot(gs[0, 1]) # Top right corner
ax2 = fig.add_subplot(gs[1, :]) # Bottom, span entire width

ax0.plot()
ax1.plot()
ax2.plot()
```

helpful parameters:

- `sharex/sharey = bool` forces plots to display the same range along the x-axis or y-axis depending upon which parameter you call. Default = False

```
In [15]: # We just need to add the ax = axes[int] parameter

# Lets make 2 distribution plots of age and applications and
# a scatter plot of number of applications by age

fig = plt.figure()
gs= fig.add_gridspec(nrows=2, ncols=2)
ax0 = fig.add_subplot(gs[0, 0]) # Top left corner
ax1 = fig.add_subplot(gs[0, 1]) # Top right corner
ax2 = fig.add_subplot(gs[1, :]) # Bottom, span entire width

ax0.hist(adopt_apps, bins=8)
```

```
ax1.hist(age_years, bins=8)  
ax2.scatter(x = age_years, y = adopt_apps)
```

Out[15]: <matplotlib.collections.PathCollection at 0x1ac483aa6f0>

